

Strojové učenie pre generáciu grafov s daným stupňom a obvodom

Machine Learning for Generation of Graphs of Given Degree and Girth

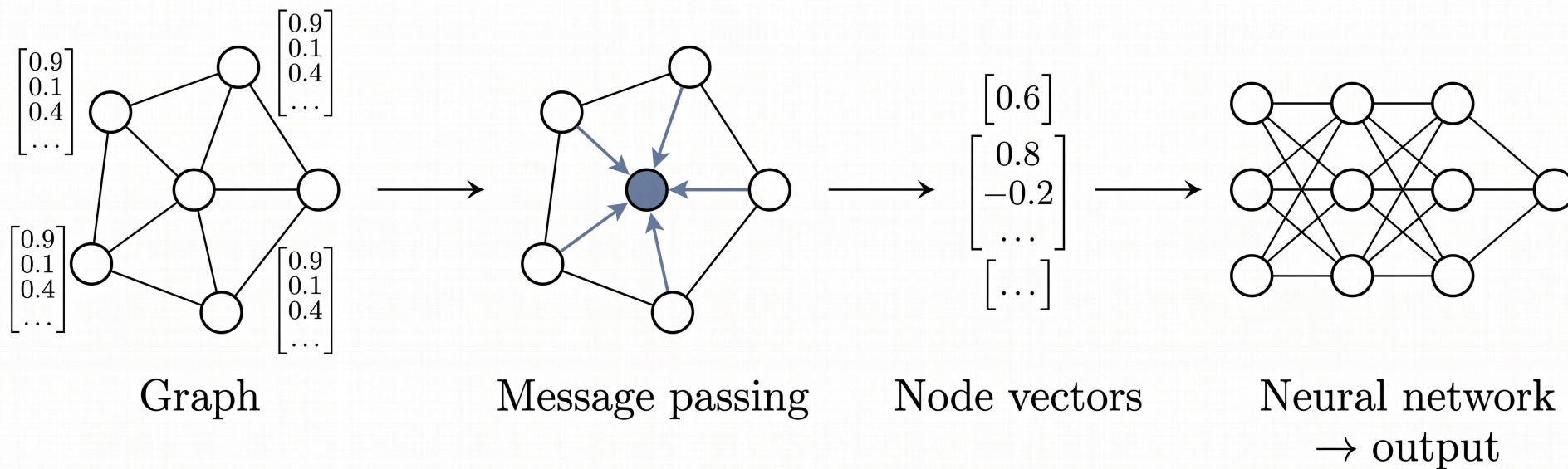
Vladimír Jančár

Mgr. Ján Pastorek

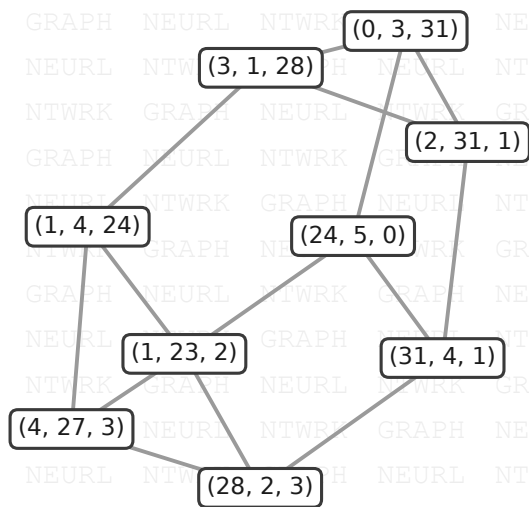
23. 06. 2026

Univerzita Komenského v Bratislave

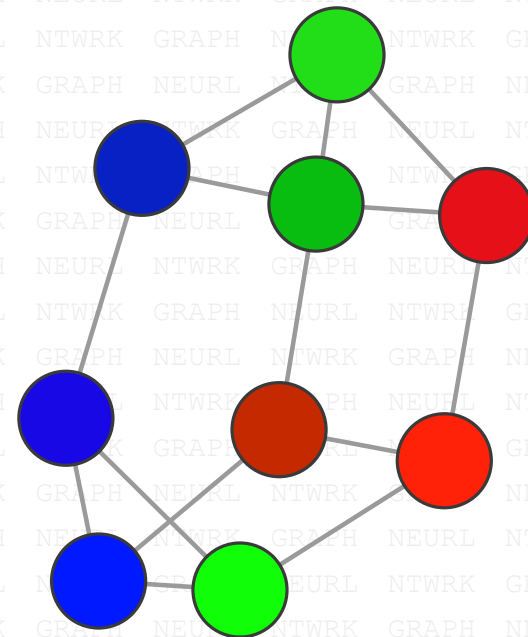
Grafové neurónové siete



Grafové neurónové siete

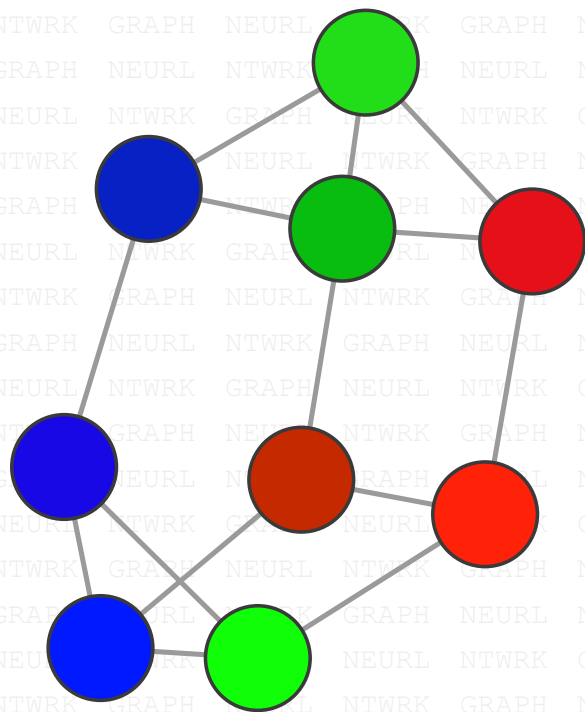


Každý vrchol má vektor

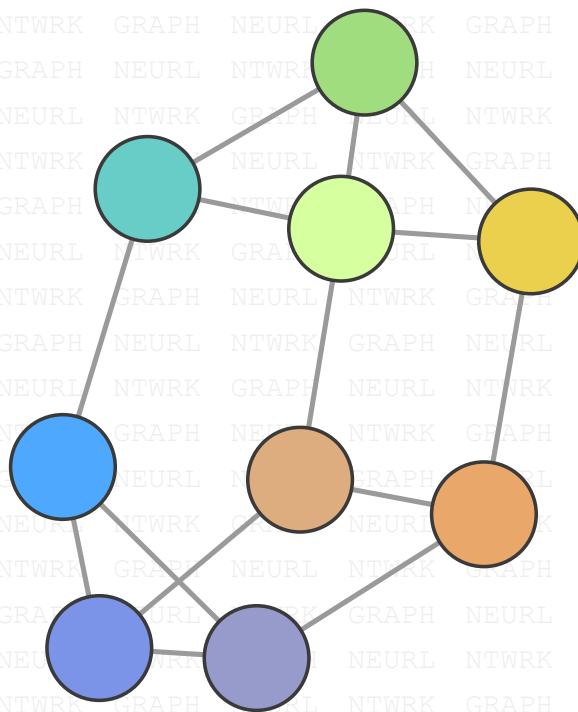


Zobrazme ako farby (podľa RGB)

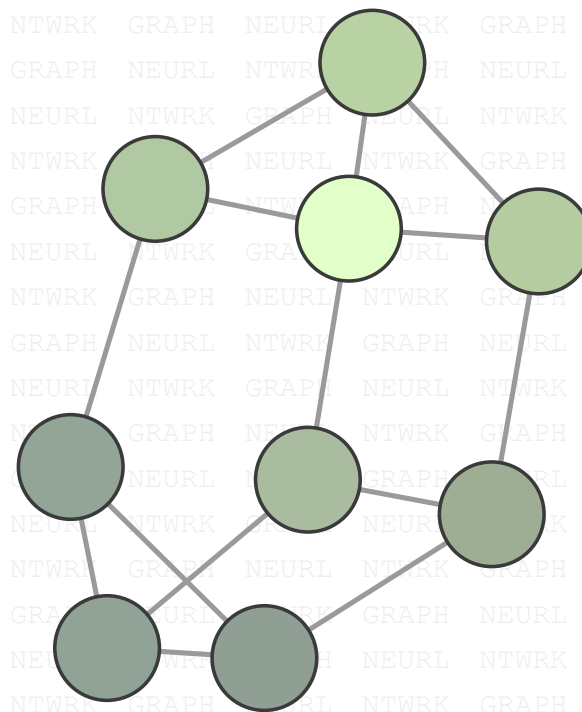
Grafové neurónové siete



začiatok

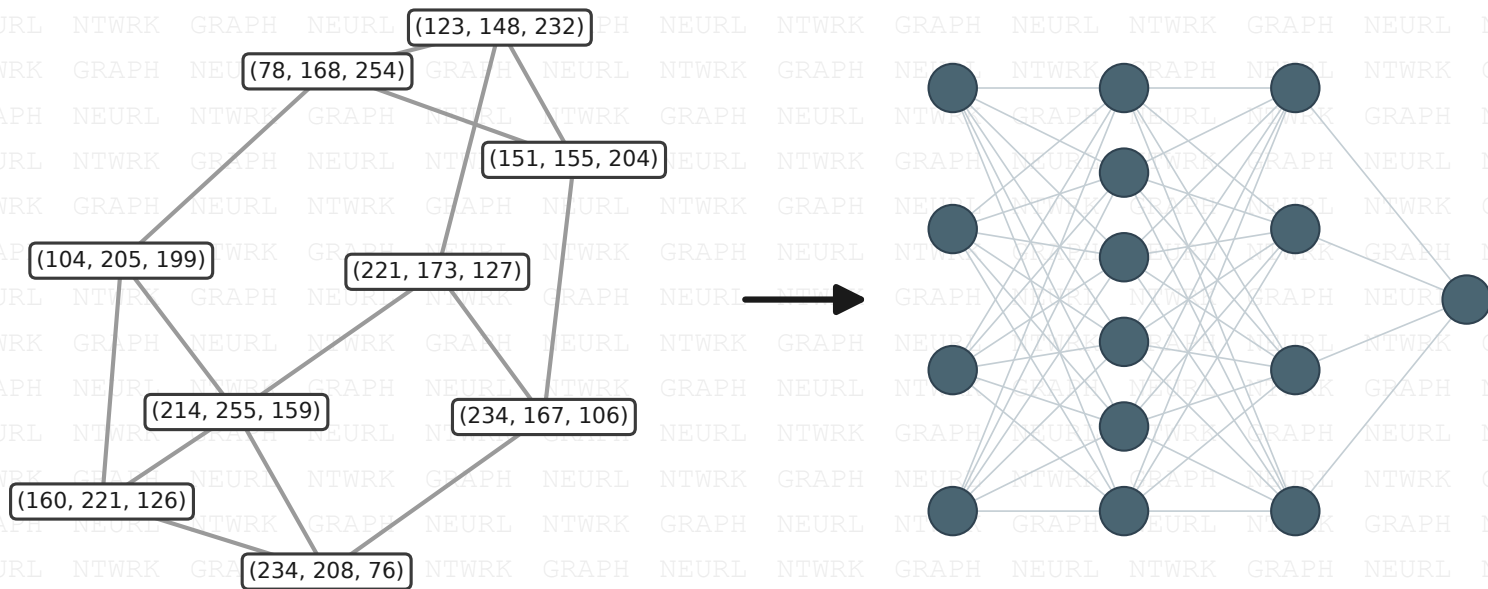


po 2 kolách



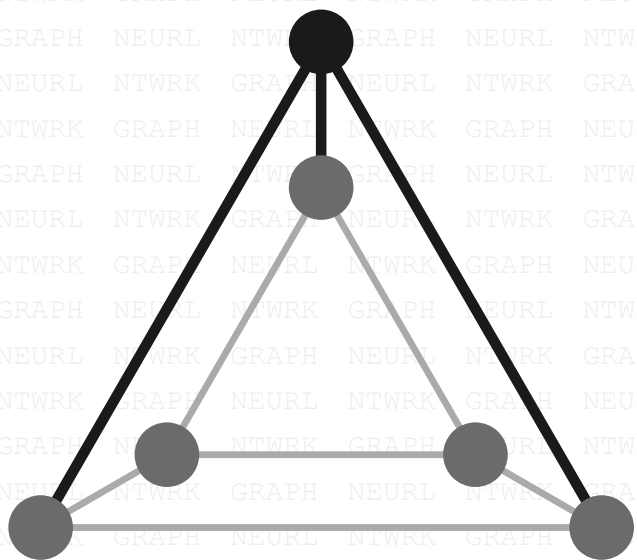
po 8 kolách

Grafové neurónové siete

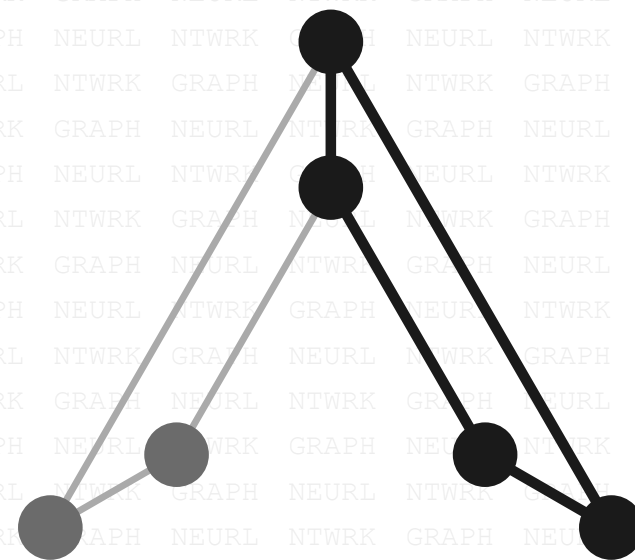


Po message-passingu vektory zachytávajú aj stav svojho okolia

Regularita a Obvod

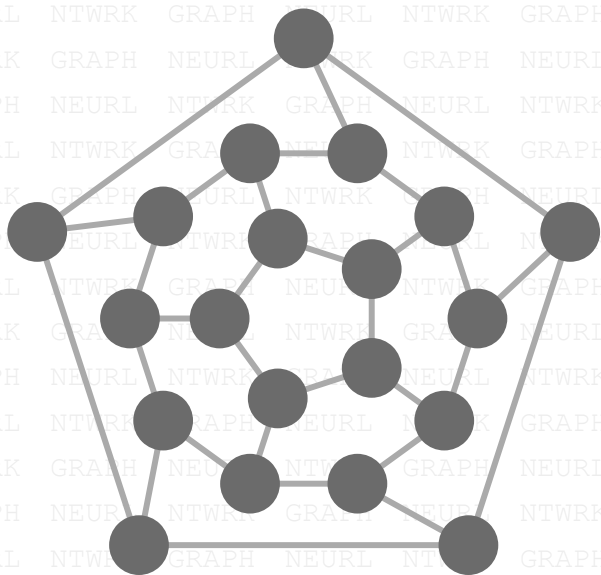


Graf je **k -regulárny**, ak každý jeho vrchol má práve **k** susedov



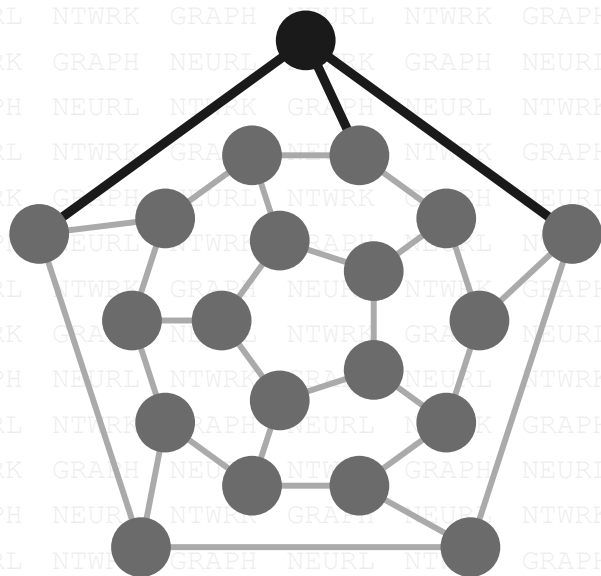
Obvod grafu **g** je veľkosť najkratšieho cyklu v grafe

(k,g)-grafy



Toto je (3,5)-graf

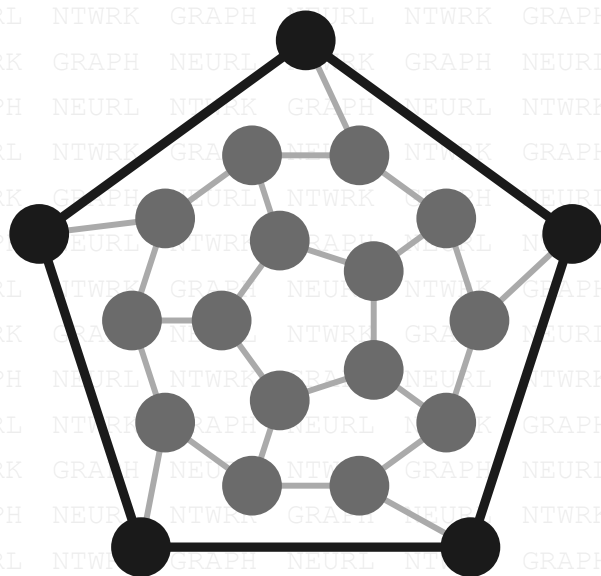
(k,g) -grafy



Toto je $(3,5)$ -graf

lebo je **3**-regulárny

(k,g) -grafy

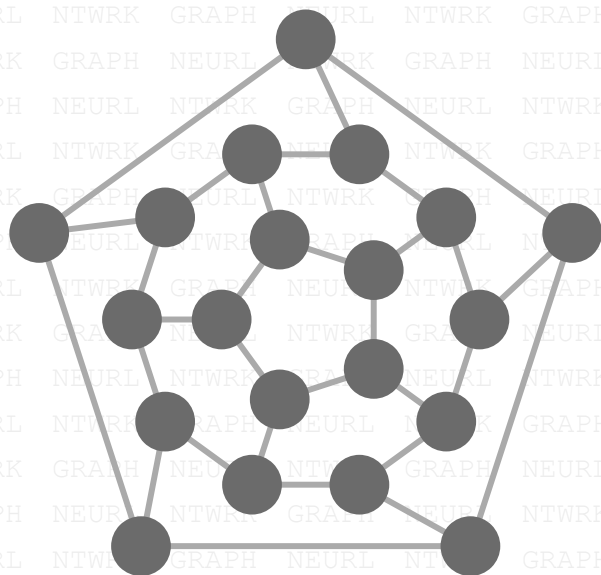


Toto je $(3,5)$ -graf

lebo je **3**-regulárny

a jeho obvod je **5**

(k,g) -grafy



Toto je $(3,5)$ -graf

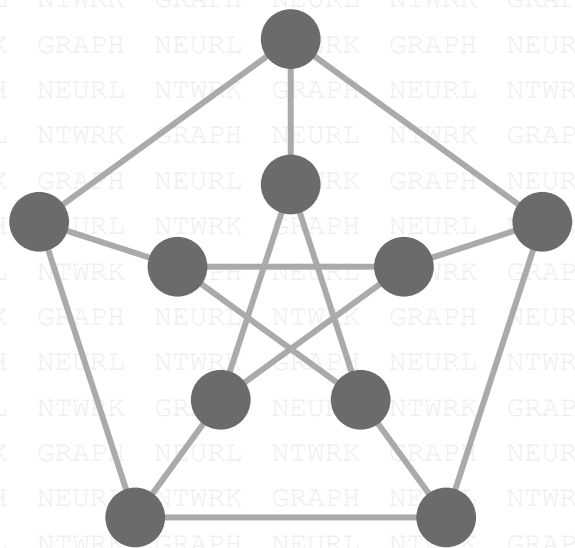
lebo je **3**-regulárny

a jeho obvod je **5**

Ale **nie** je klietka

- najmenší možný (k,g) -graf

(k,g) -grafy



Toto je tiež $(3,5)$ -graf

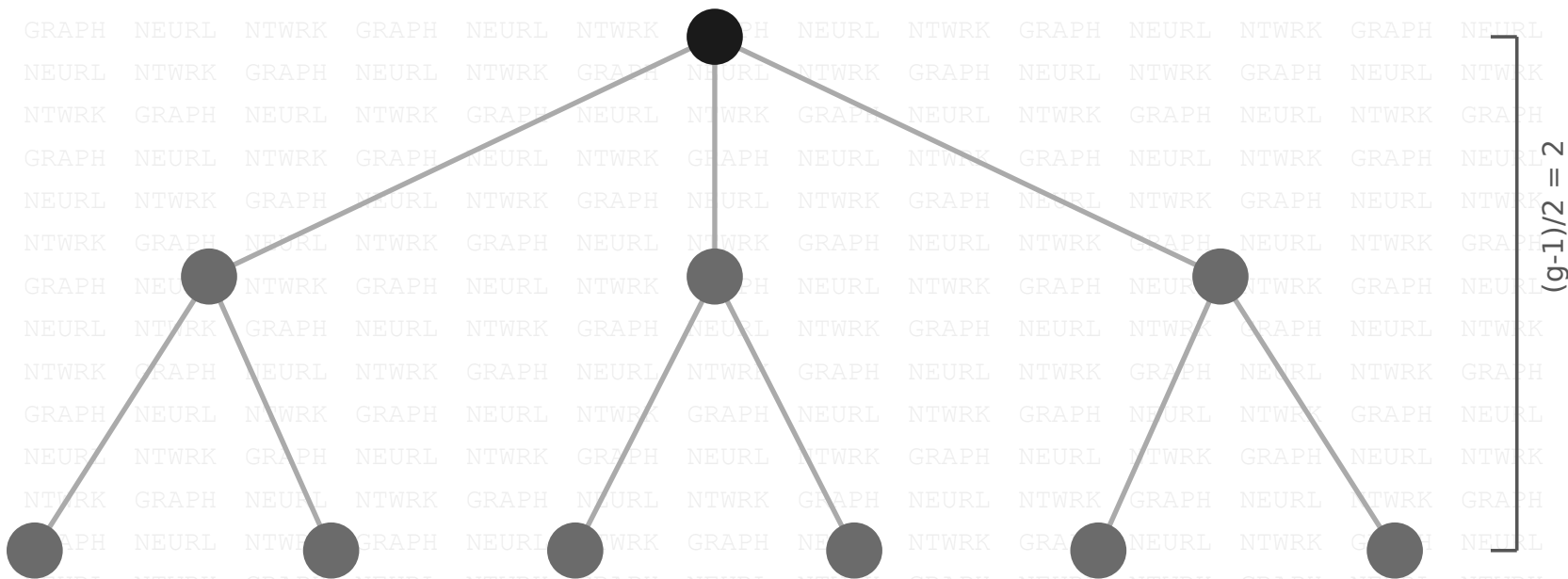
a je to **klietka**

Moore's bound

1

3

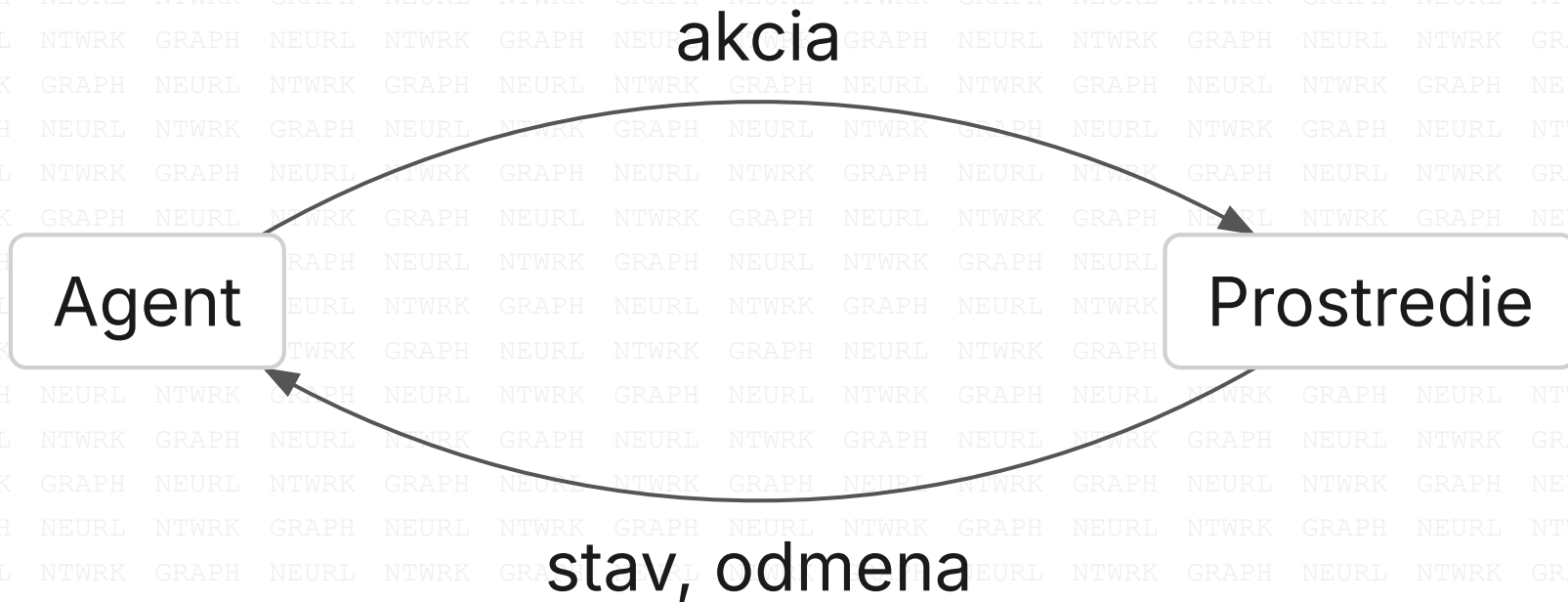
6



$$1 + 3 + 6 = 10$$

Najmenší počet vrcholov, ktorý potrebujeme na (k, g) -graf

Reinforcement learning

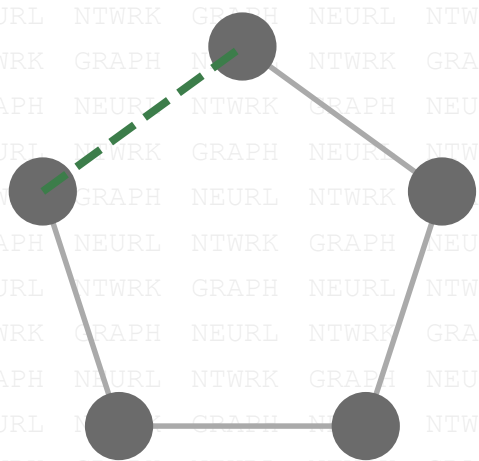


Agent stavia graf hranu po hrane
Dostáva odmenu a z nej sa učí

Reinforcement learning

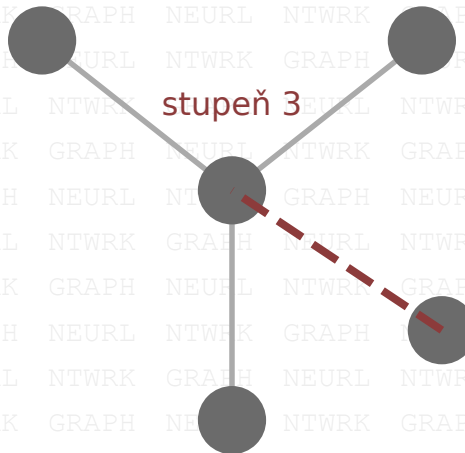
✓ **legálny ťah**

uzavrie 5-cyklus



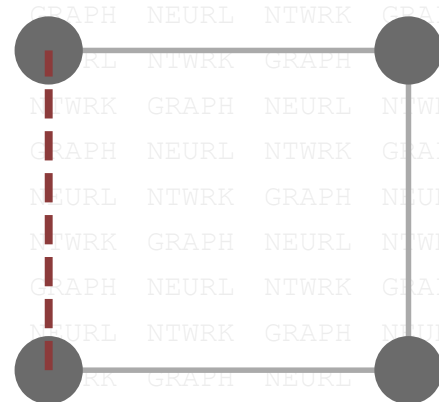
✗ **porušený stupeň**

vrchol by mal stupeň $> k$



✗ **porušený obvod**

vznikol by 4-cyklus $< g$



Akcia je **toggle** hrany (u, v) : pridá sa, ak chýba, odoberie sa, ak tam je

Reinforcement learning

Agentov cieľ: **k-regulárny graf s obvodom aspoň g**

+50 za hotový graf

+0,05 keď sa graf stane k-regulárnym alebo dosiahne obvod g

+0,01 za pridanie hrany

-0,02 za odobratie hrany

K tomu priebežná odmena za postup k cieľu

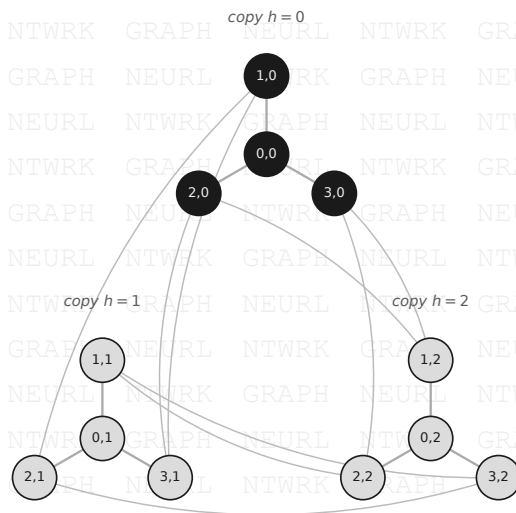
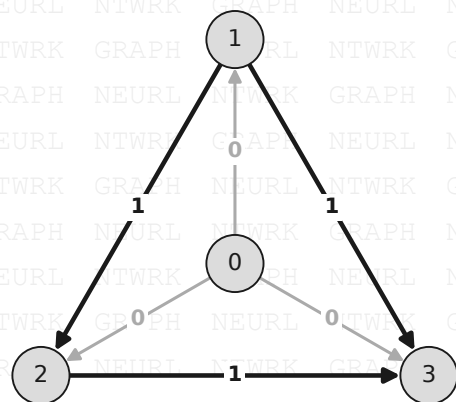
Reinforcement learning: výsledky

RL zvládlo len najmenšie ciele: (5,3), (3,5), (6,3) a podobné

Úspešnosť 8 %, a zo všetkých metód bolo najpomalšie

GRAPH NEURL NTWRK GRAPH NEURL NTWRK GRAPH NEURL NTWRK GRAPH NEURL NTWRK 7 malébo grafu spraví veľký a GRAPH NEURL NTWRK

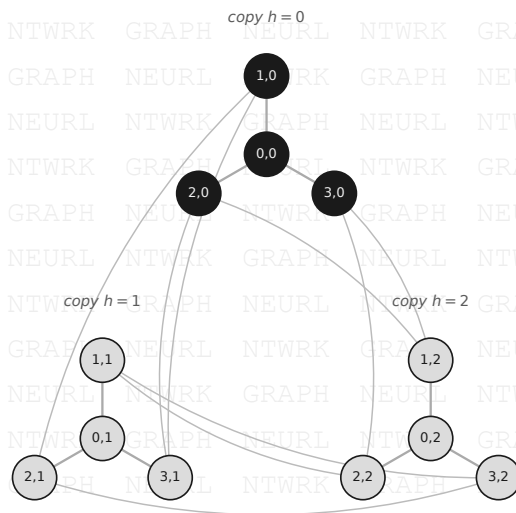
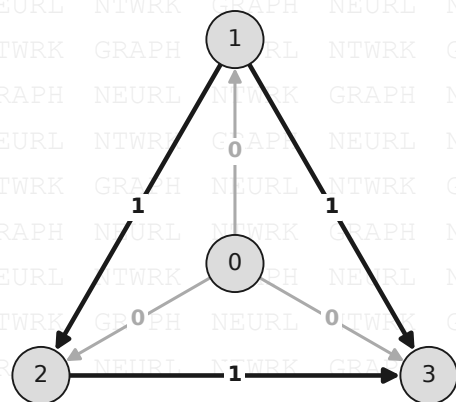
Voltage lifts



Z malého grafu spraví veľký a zachová regularitu

Na lifte potrebujeme:

Voltage lifts

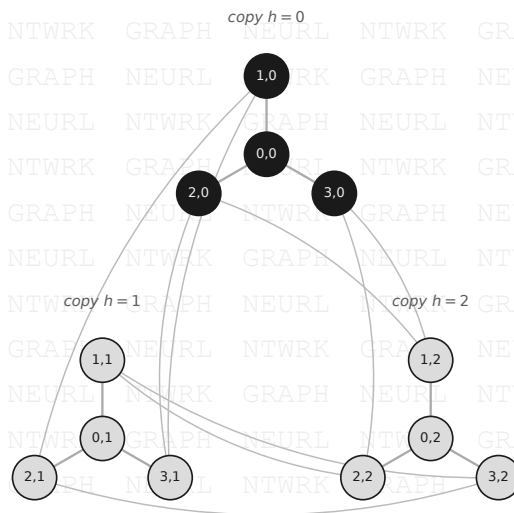
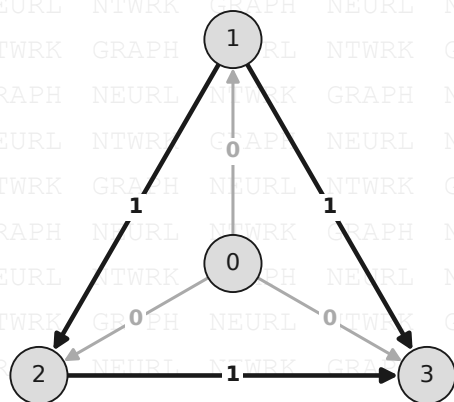


Z malého grafu spraví veľký a zachová regularitu

Na lifte potrebujeme:

Zvoliť základný graf G a grupu Γ

Voltage lifts



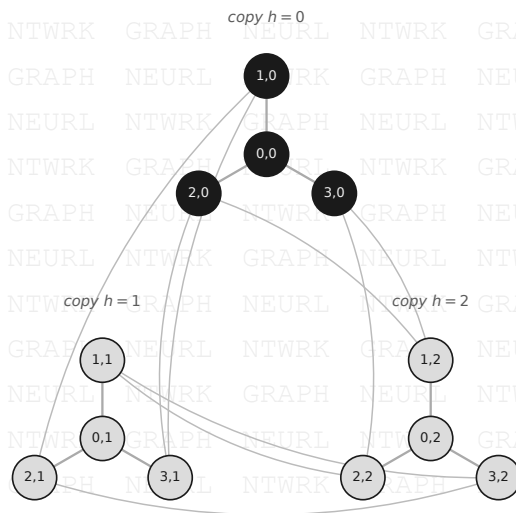
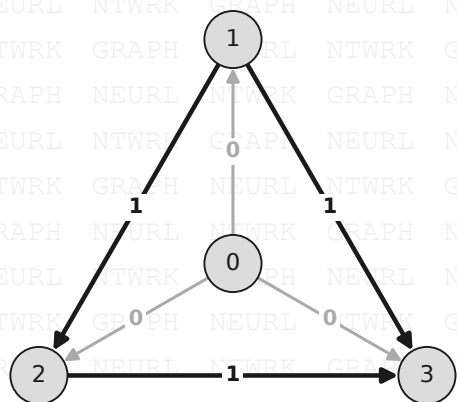
Z malého grafu spraví veľký a zachová regularitu

Na lifte potrebujeme:

Zvoliť základný graf G a grupu Γ

Priradiť jej prvky hranám - napätia

Voltage lifts



Z malého grafu spraví veľký a zachová regularitu

Na lifte potrebujeme:

Zvoliť základný graf G a grupu Γ

Priradiť jej prvky hranám - napätia

$$|V(\text{lift}(G))| = |\Gamma| \times |V(G)|$$

Voltage lifts



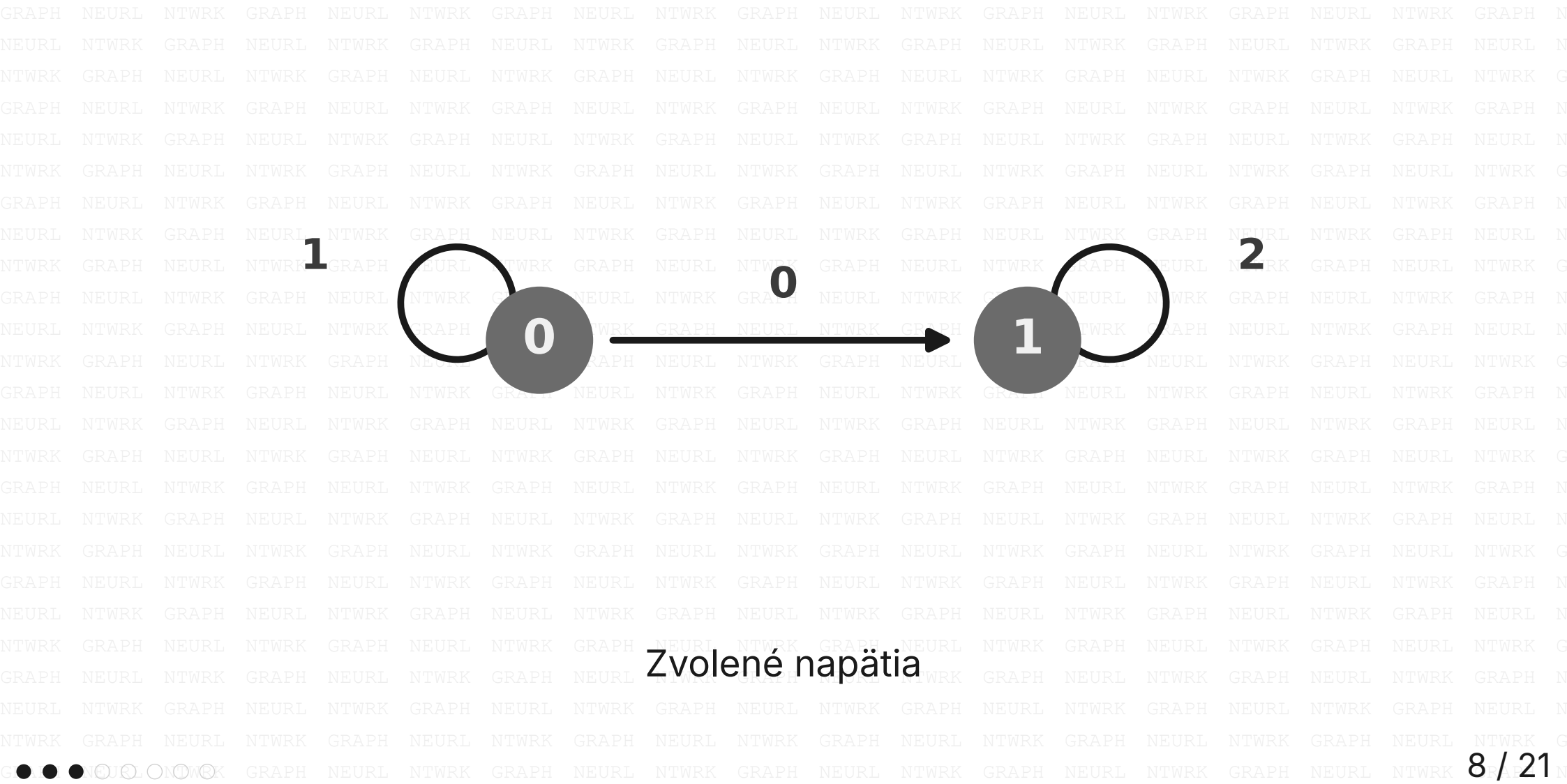
Základný graf G

Voltage lifts

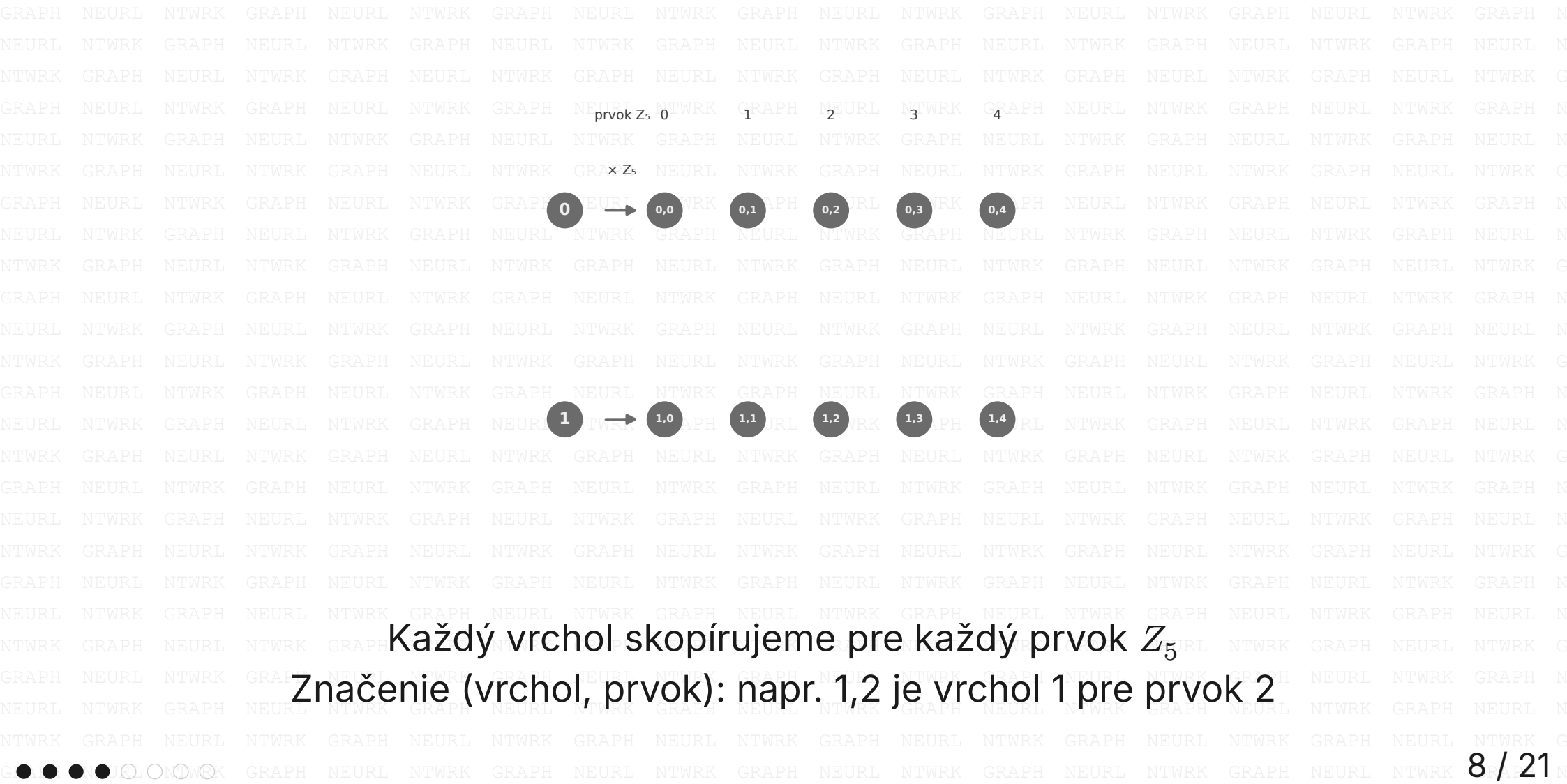


Zvolená grupa: $Z_5 = \{0, 1, 2, 3, 4\}$

Voltage lifts



Voltage lifts

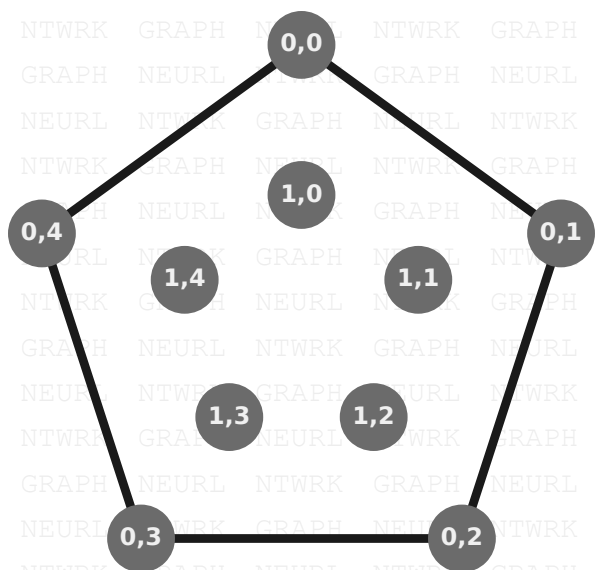


Voltage lifts



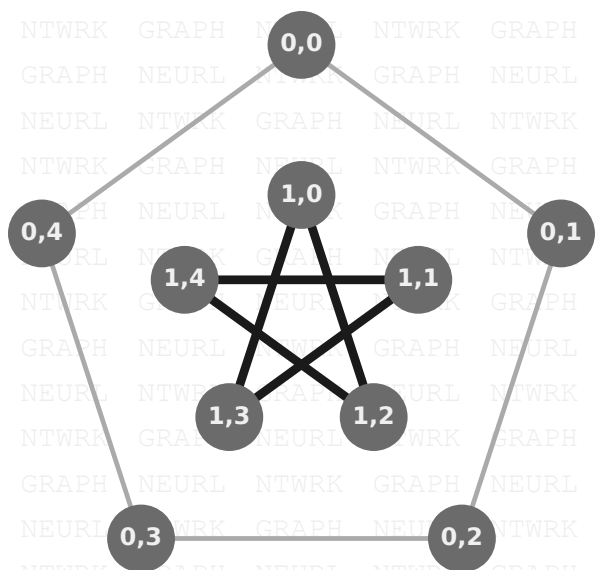
Pre prehľadnosť zoradíme vrcholy takto.

Voltage lifts



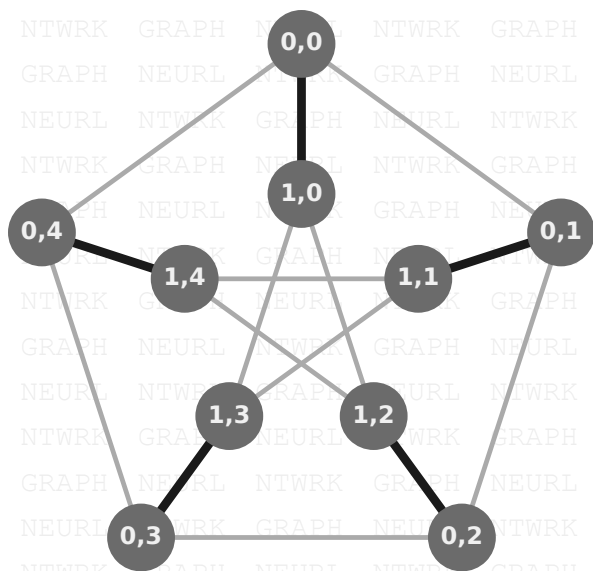
Slučka na 0 dá hrany $(0,x) \rightarrow (0,x+1)$

Voltage lifts



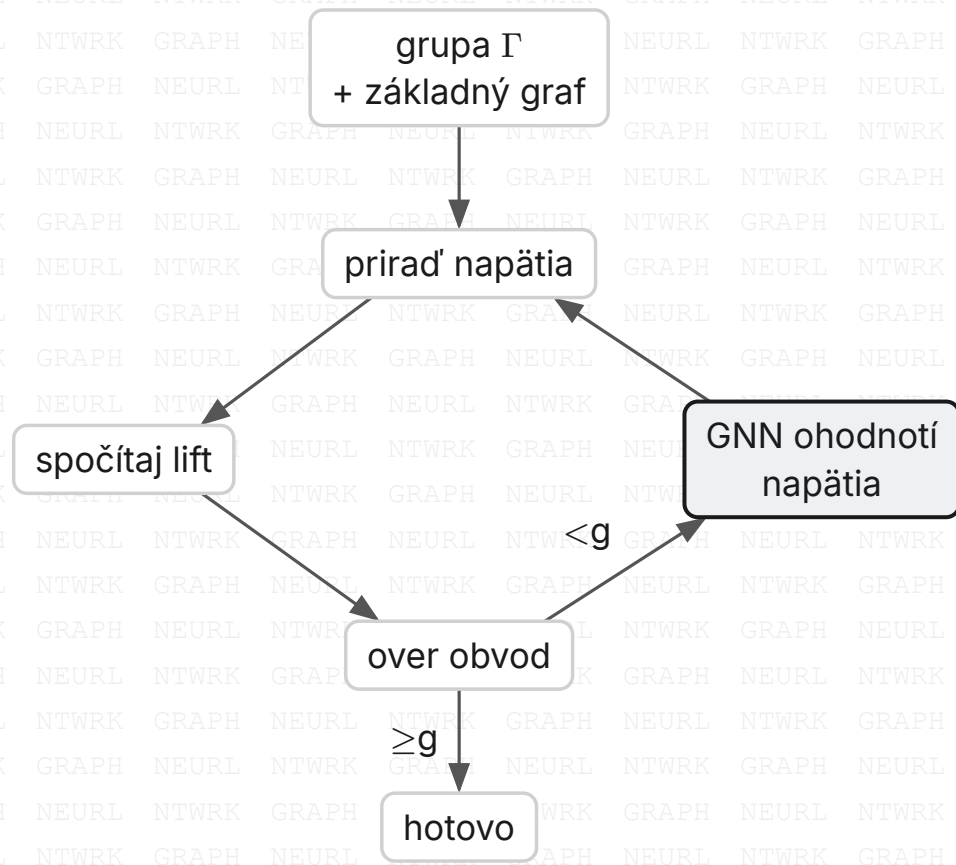
Slučka na 1 dá hrany $(1,x) \rightarrow (1,x+2)$

Voltage lifts



Hrana $0 \rightarrow 1$ dá hrany $(0,x) \rightarrow (1,x)$. Výsledek je Petersenov graf

Voltage lifts a GNN

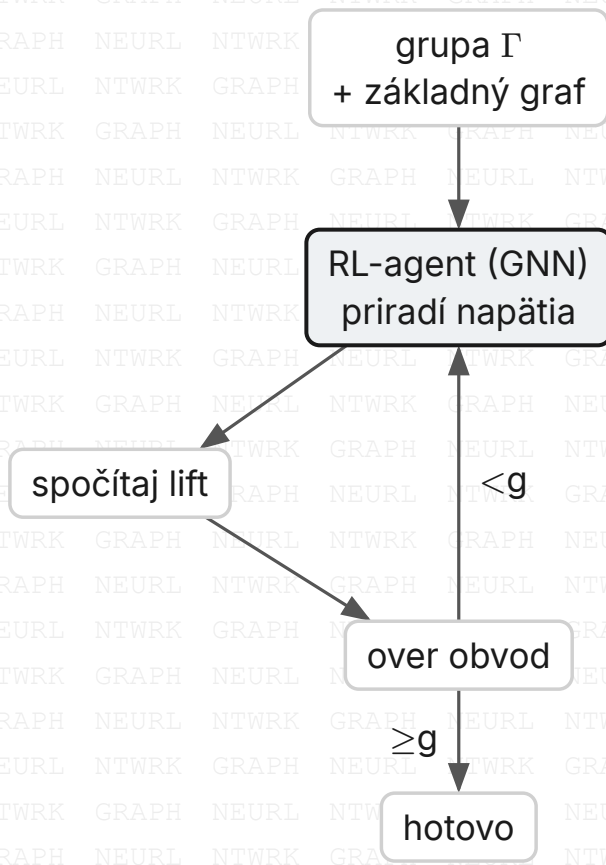


Bez GNN: napätia
priradzujeme náhodne a po
každom lifte kontrolujeme
obvod

GNN predikuje obvod liftu
podľa daných napätí

bez GNN 63 %,
s GNN 59 %
GNN tu nepomohlo

Voltage lifts a GNN



GNN RL-agent priraduje
napätia hranám

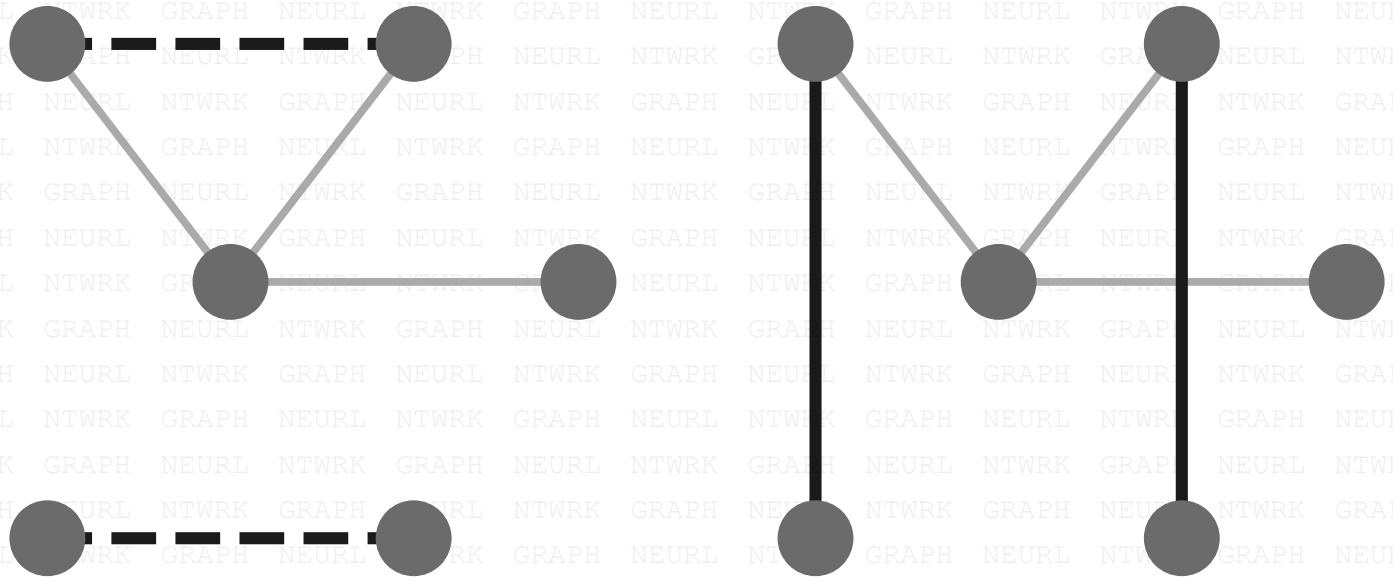
Tento prístup neskôr
nazývam **voltage_rl**

Najširší dosah zo všetkých
metód: 84 %

Skoro-dobrý graf, ale ostali krátke cykly

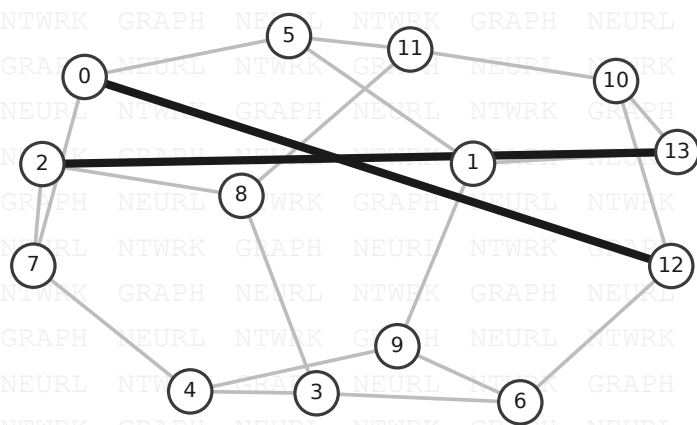
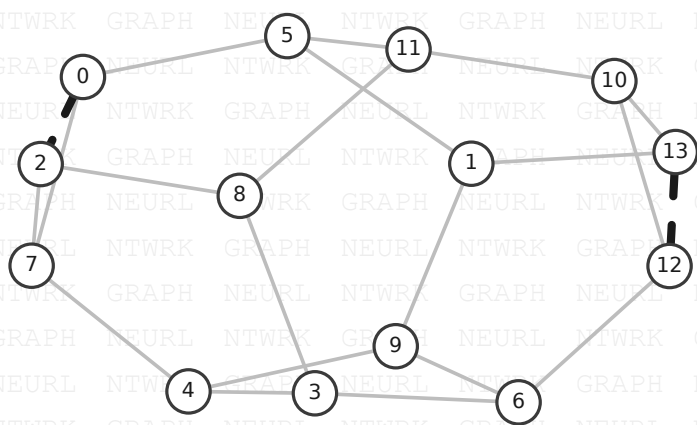
10 / 21

Refinement



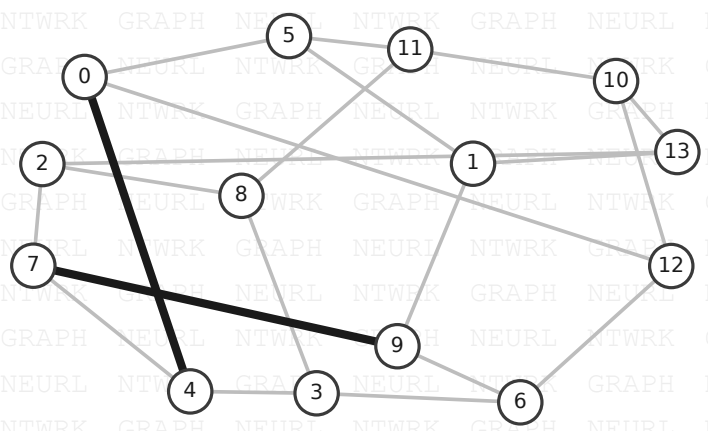
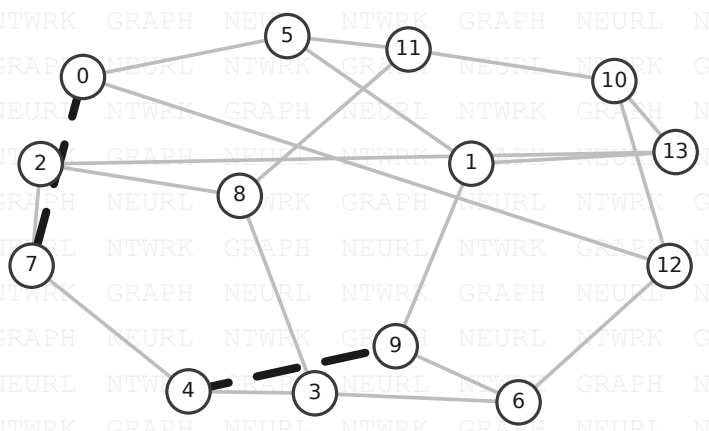
2-swap: zachováme stupeň, vymažeme krátky cyklus

Refinement



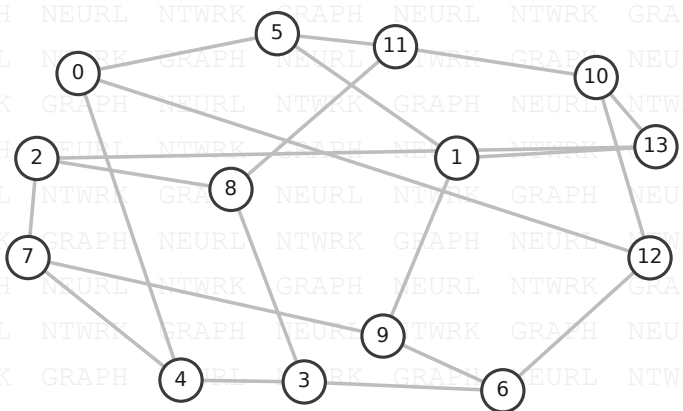
Prvý 2-swap: odoberieme dve hrany (vľavo), pridáme dve nové (vpravo)

Refinement



Druhý 2-swap: odoberieme (vľavo), pridáme (vpravo)

Refinement



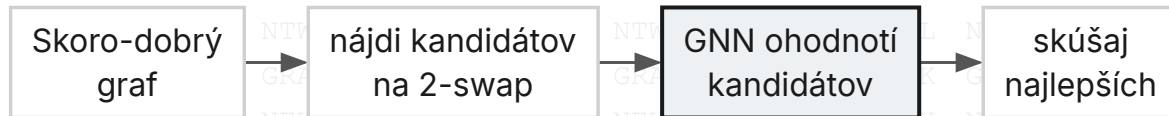
Po dvoch swapoch má graf obvod 5

Refinement a GNN

Bez GNN: náhodne skúšame
kandidátov na 2-swap



Refinement a GNN



Bez GNN: náhodne skúšame kandidátov na 2-swap

GNN zoraduje kandidátov podľa pravdepodobnosti, že swap zvýši obvod

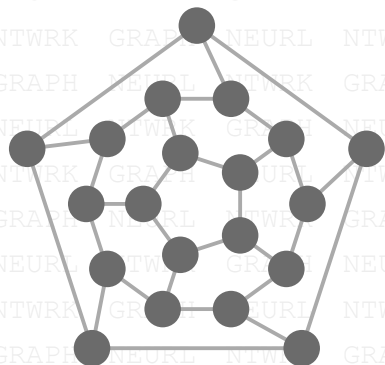
Refinement a GNN



Bez GNN: náhodne skúšame kandidátov na 2-swap

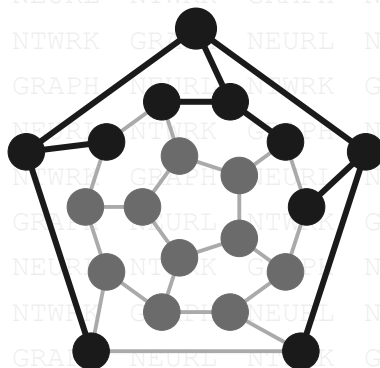
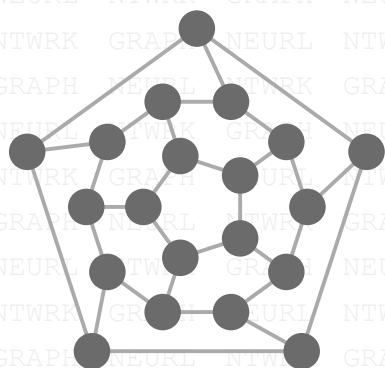
GNN zoraduje kandidátov podľa pravdepodobnosti, že swap zvýši obvod

Úspešnosť sa znížila
51% s GNN vs. 75% bez GNN



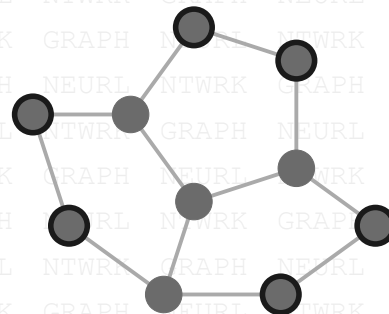
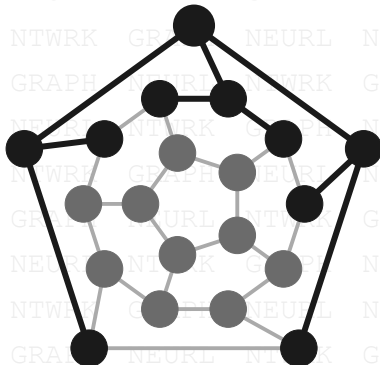
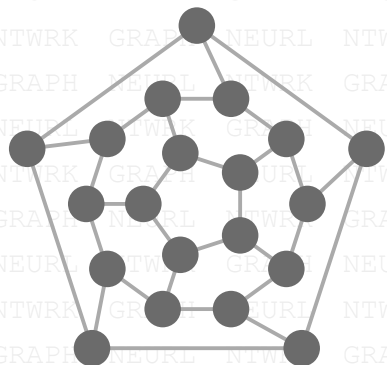
Začneme s priveľkým (3,5)-
grafom: dodekaéder, 20
vrcholov

Excízia



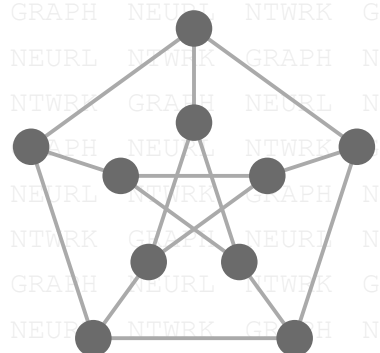
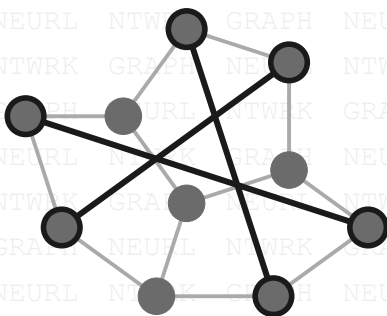
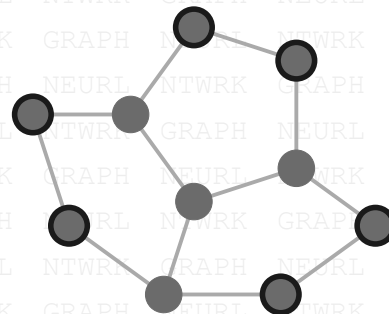
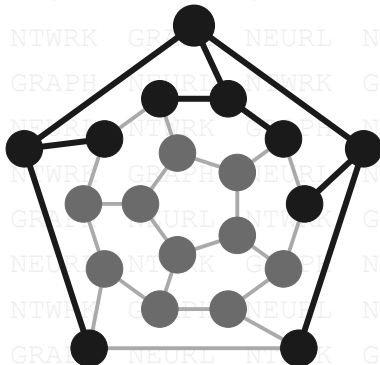
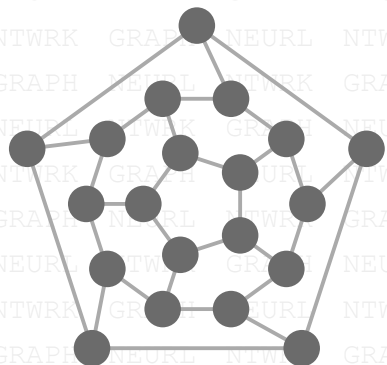
Zakoreníme strom hĺbky 2 -
Moorov polomer $\left\lfloor \frac{g-1}{2} \right\rfloor$

Excízia



Strom vyrežeme, ostane 10
vrcholov, obvod 5, no už
nie je k -regulárny

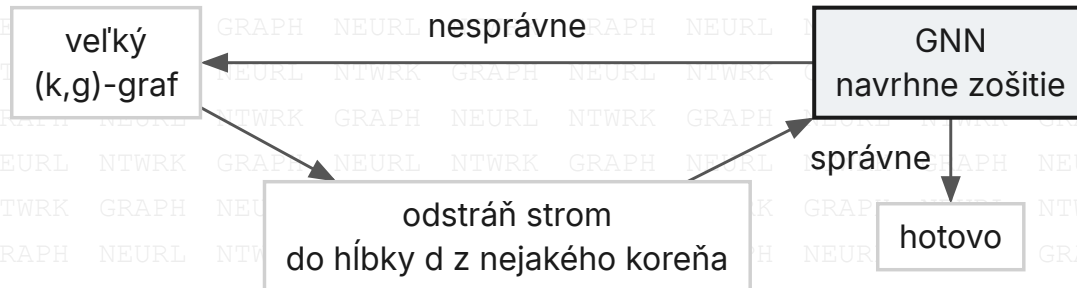
Excízia



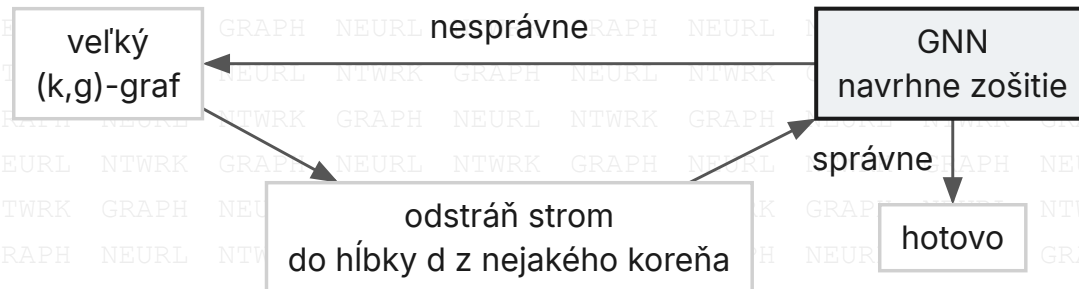
Výsledok je Petersenov
graf, teda (3,5)-klietka

Excízia a GNN

GNN navrhuje ktoré vrcholy
zošit.



Excízia a GNN



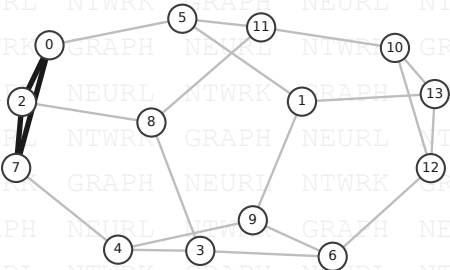
GNN navrhuje ktoré vrcholy zošit'.

Bez excízie $2,06\times$ Moore, s ňou $1,51\times$ GNN excíziu zrýchlilo

**Z voltage lifts, refinementu a excízie
sme vytvorili jednu pipeline**

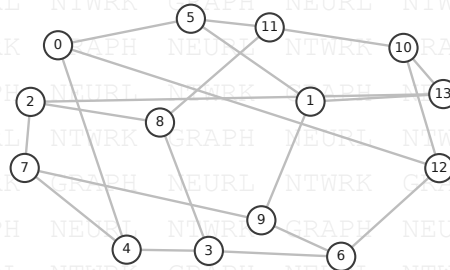
Forge

Producer



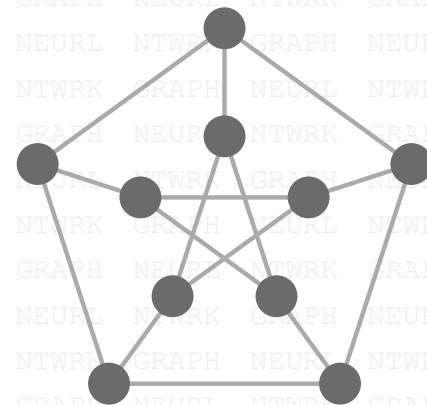
skoro-dobry lift

Refinement



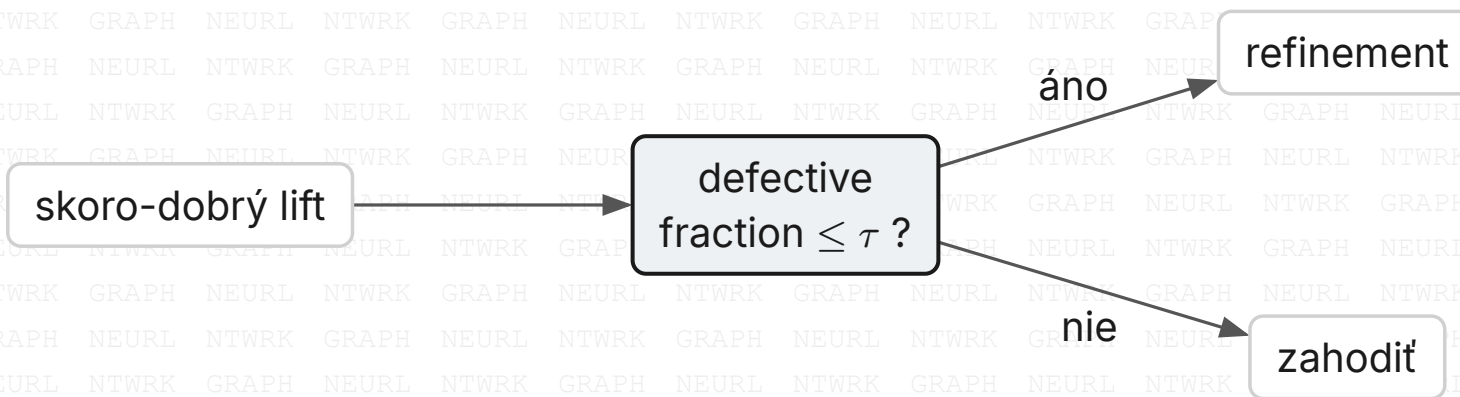
obvod = g

Excízia



zmenšený (k,g) -graf

Forge

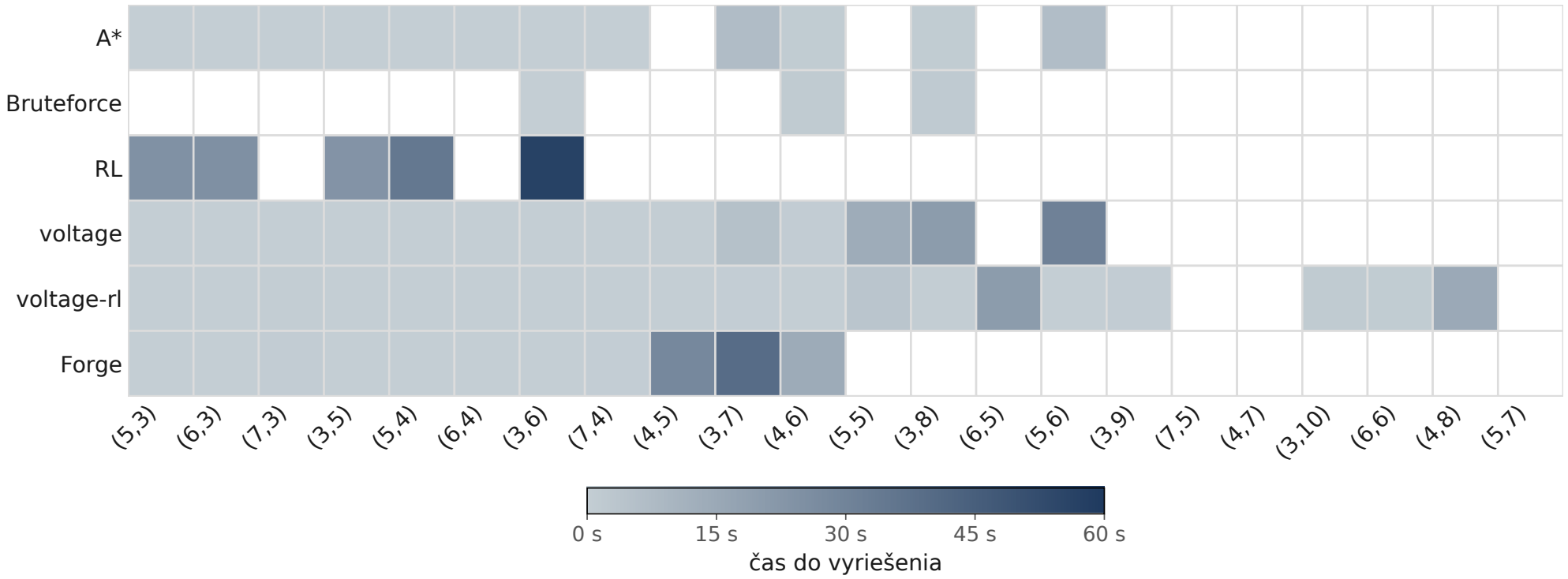


$$\text{defective fraction} = \frac{|E_{\text{def}}|}{|E|}$$

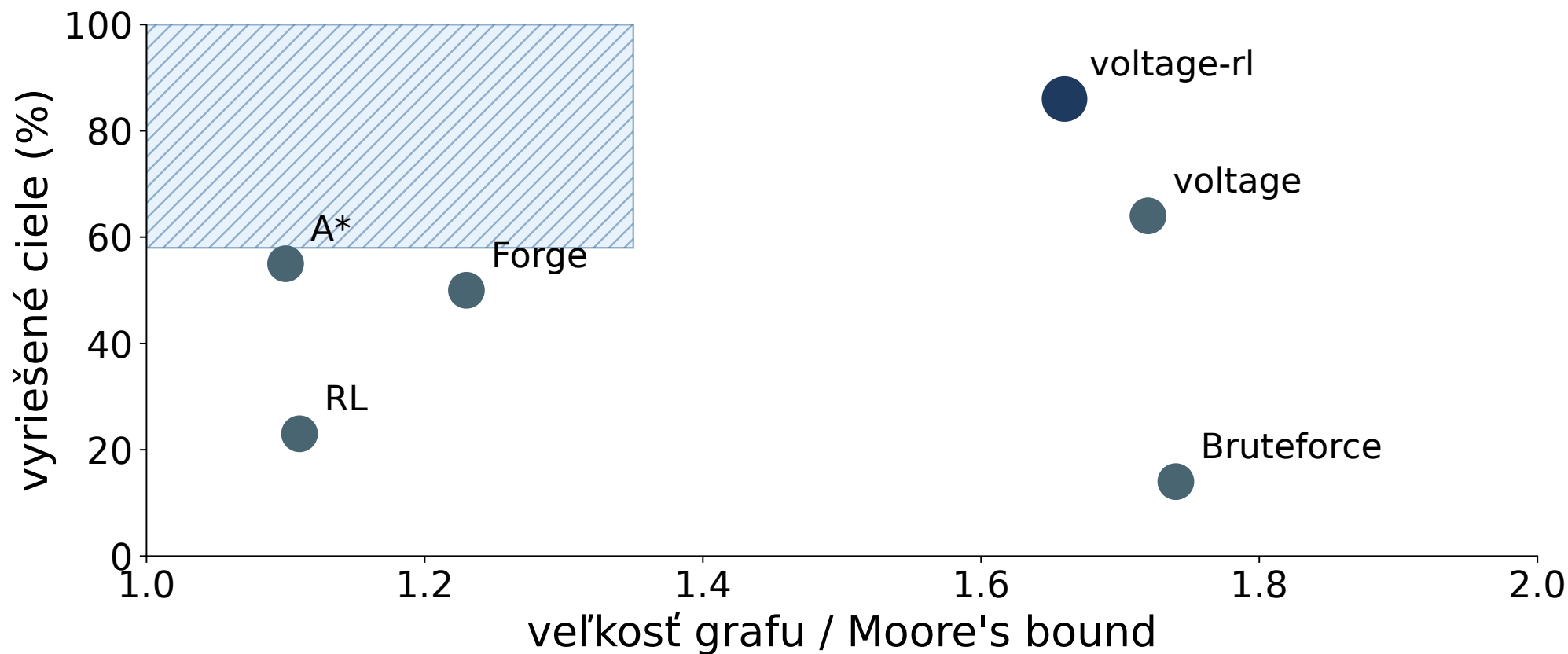
E_{def} = hrany ležiace na cykle kratšom ako g , teda tie, čo kazia obvod

Brána pustí na opravu len lify s malým podielom chybných hrán

Výsledky



Výsledky



Otázka školiteľa (1)

Čakal som, že kombinovaná metóda Forge dosiahne najlepšie výsledky. Podľa tabuliek vychádza najlepšie A* pri hľadaní najmenšieho príkladu (k,g) -grafu a voltage-RL pri hľadaní aspoň nejakého príkladu. Čo zapríčinilo dané výsledky?

Otázka školiteľa (1)

Otázka školiteľa (1)

Otázka školiteľa (1)

Forge mal byť najlepším - pokus o najlepšiu metódu, keď samotné metódy nedávali výsledky, aké som chcel
voltage-RL má lepší dosah - prejde za rovnaký čas viac grafov

metóda	grafov/s
Forge	≈ 26
voltage-RL	≈ 40
Forge bez refinementu	≈ 65

pri hľadaní (3,12)

Otázka školiteľa (1)

Forge mal byť najlepším - pokus o najlepšiu metódu, keď samotné metódy nedávali výsledky, aké som chcel

voltage-RL má lepší dosah - prejde za rovnaký čas viac grafov

A* nájde menšie grafy - Je best-first, uprednostňuje malé grafy a k veľkým sa ani nedostane. Forge producer môže vytvoriť veľký (k,g)-graf, ktorý excission nezmenší

metóda	grafov/s
Forge	≈ 26
voltage-RL	≈ 40
Forge bez refinementu	≈ 65

pri hľadaní (3,12)

Otázka školiteľa (1)

Forge mal byť najlepším - pokus o najlepšiu metódu, keď samotné metódy nedávali výsledky, aké som chcel

voltage-RL má lepší dosah - prejde za rovnaký čas viac grafov

A* nájde menšie grafy - Je best-first, uprednostňuje malé grafy a k veľkým sa ani nedostane. Forge producer môže vytvoriť veľký (k,g)-graf, ktorý excission nezmenší

Najväčší problém robí refinement -

Pravdepodobne dostáva zlých kandidátov.

Producer je trénovaný na produkciu dobrých grafov, nie skoro-dobrých

metóda	grafov/s
Forge	≈ 26
voltage-RL	≈ 40
Forge bez refinementu	≈ 65

pri hľadaní (3,12)

Otázka školiteľa (1)

Forge mal byť najlepším - pokus o najlepšiu metódu, keď samotné metódy nedávali výsledky, aké som chcel

voltage-RL má lepší dosah - prejde za rovnaký čas viac grafov

A* nájde menšie grafy - Je best-first, uprednostňuje malé grafy a k veľkým sa ani nedostane. Forge producer môže vytvoriť veľký (k,g)-graf, ktorý excission nezmenší

Najväčší problém robí refinement -

Pravdepodobne dostáva zlých kandidátov.

Producer je trénovaný na produkciu dobrých grafov, nie skoro-dobrých

Future work: producer učení na skoro-dobré grafy

metóda	grafov/s
Forge	≈ 26
voltage-RL	≈ 40
Forge bez refinementu	≈ 65

pri hľadaní (3,12)

Otázka školiteľa (2)

Prečo ste metódy zastavili po 60 sekundách? Ako vieme, či modely so strojovým učeníím nepotrebovali viac času na konvergenciu?

Otázka školiteľa (2)

Najprv som benchmarkoval na 5 minútach

Otázka školiteľa (2)

Najprv som benchmarkoval na **5 minútach**
Ak sa podarí nájsť graf, podarí sa to **do 1 minúty**. Ak nie, **ani za 5**

Otázka školiteľa (2)

Najprv som benchmarkoval na **5 minútach**

Ak sa podarí nájsť graf, podarí sa to **do 1 minúty**. Ak nie, **ani za 5**

Neskôr som benchmarky len rozširoval a vyhradený čas už nemenil

Otázka školiteľa (2)

Najprv som benchmarkoval na **5 minútach**

Ak sa podarí nájsť graf, podarí sa to **do 1 minúty**. Ak nie, **ani za 5**

Neskôr som benchmarky len rozširoval a vyhradený čas už nemenil

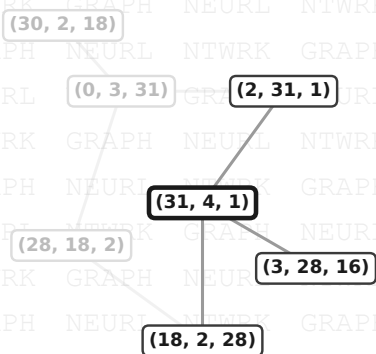
Modely **konvergujú pri tréningu**, nezávisle od tohto času.

Výsledky sú z benchmarkov **po** tréningu

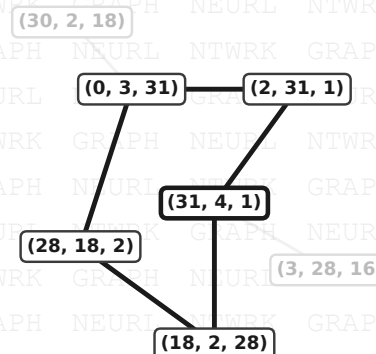
Otázka oponenta (1)

V práci ukazujete, že štandardné GNN nedokážu spoľahlivo predikovať obvod. Aké štrukturálne alebo pozičné príznaky by ste do modelu pridali, aby lepšie zachytil globálnu cyklickú štruktúru grafu?

Otázka oponenta (1)



Štandardné GNN vidí len susedov



Loopy vidí aj slučku okolo v

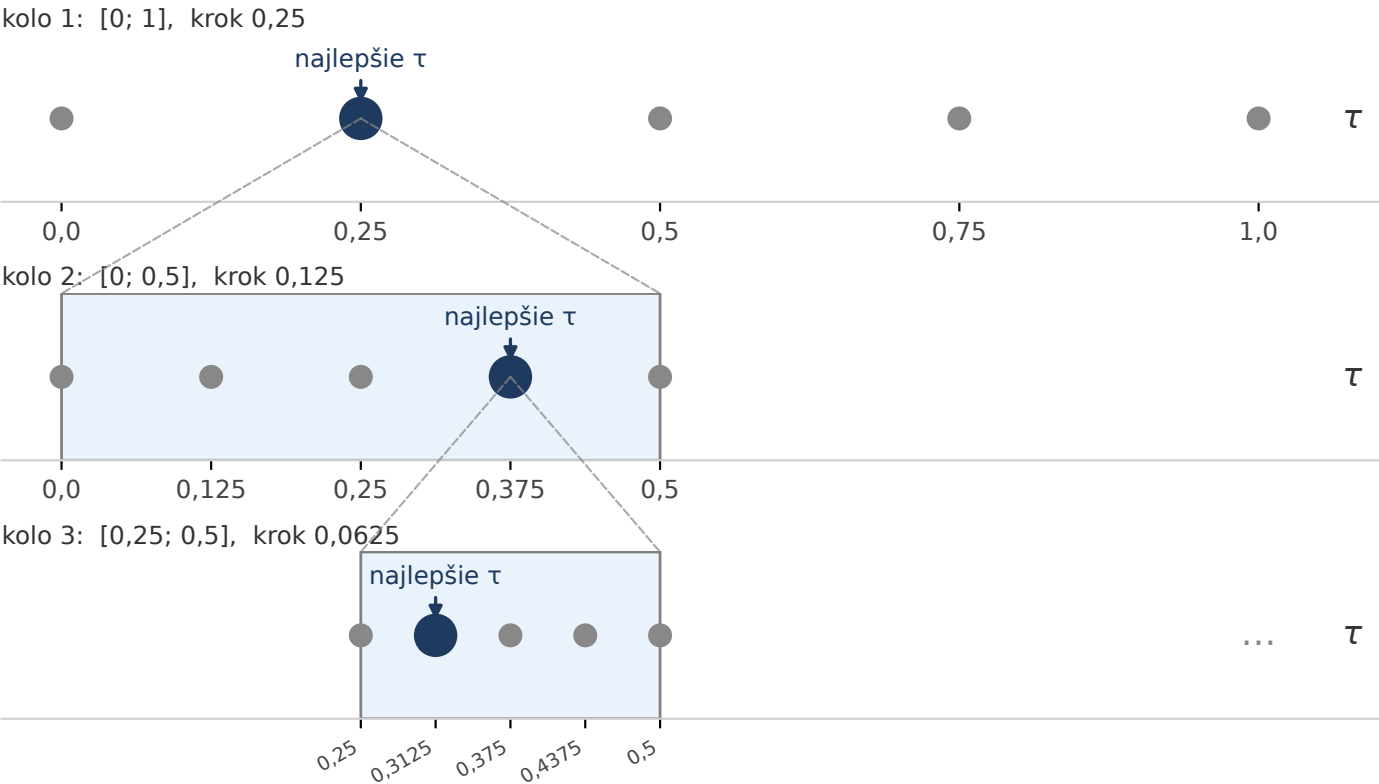
Inšpiroval by som sa architektúrou **Loopy**, ktorá sa to snaží riešiť: agreguje aj po slučke okolo vrcholu, a tak jeho vektor lepšie zachytí, na akom cykle leží.

Otázka oponenta (2)

Pipeline Forge kombinuje voltage producer, refinement a excíziu. Ako by ste systematicky ladili prah defective fraction a ako by ste experimentálne odlišili prínos diverzity kandidátov od kvality naučenej politiky?

Otázka oponenta (2)

Systematické ladenie prahu τ

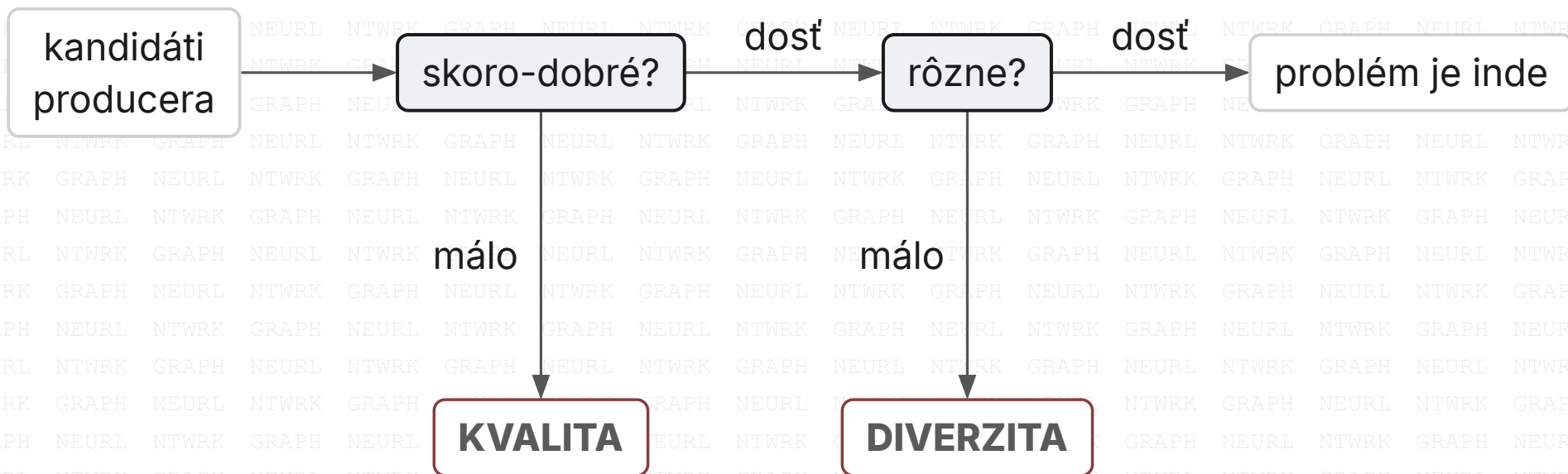


Otázka oponenta (2)

**Diverzita vs. kvalita producera: voltage-RL je samostatne nejlepší,
ale ako Forge producer najslabší**

Otázka oponenta (2)

Diverzita vs. kvalita producera: voltage-RL je samostatne nejlepší,
ale ako Forge producer najslabší



Záver

Efektívne som implementoval metódy na generovanie (k,g)-grafov a porovnal učené verzie s neučenými

Efektívne som implementoval metódy na generovanie (k,g) -grafov a porovnal učené verzie s neučenými

Hlavné zistenie je, že učením sa oplatí navádzať hľadanie, ale neoplatí sa nechať hľadanie úplne na GNN

Efektívne som implementoval metódy na generovanie (k,g)-grafov a porovnal učené verzie s neučenými

Hlavné zistenie je, že učením sa oplatí navádzať hľadanie, ale neoplatí sa nechať hľadanie úplne na GNN

Pipeline Forge má potenciál, len je potrebné zlepšiť kvalitu refinement kroku a/alebo kandidátov

Oponentom odporúčané otázky na štátnice

5.P. Efektívne reprezentácie dátovej štruktúry graf. Využitie problému Union-find pri hľadaní kostry grafu.

10.P. Spôsoby prehľadávania stavového priestoru, do hĺbky a do šírky, Dijkstrov algoritmus. Porovnanie implementácií v rôznych jazykoch.